robstatpy: Python Wrappers for Modern Robust Statistical Estimation

Google Summer of Code 2026, Project Proposal

Organization: **R Project for Statistical Computing** · Submitted:

1. Project Title and Short Description

Title

robstatpy: Python Wrappers for the RobStatTM Robust Statistics R Package

Short Description

The RobStatTM R package (Maronna, Martin, Yohai & Salibian-Barrera, 2019) implements state-of-the-art robust estimators, including MM-regression, Rocke's S-estimator for high-dimensional covariance, robust PCA via M-scale minimization, and Bianco–Yohai logistic regression. While recent Python packages cover some robust methods, the RobStatTM-specific implementations, notably the DCML regression estimator, robust model selection via RFPE, and robust hypothesis testing, remain unavailable in Python. This project will produce **robstatpy**, a Python library exposing the full RobStatTM function set through lightweight rpy2 wrappers whose numerical outputs reproduce the R originals exactly. Every wrapper will ship with Python-style documentation modelled on the RobStatTM R man pages, a pytest validation suite, and reproducible Jupyter notebook examples.

2. Student Information

2.1. Contact Details

Field	Details
Full Name	Aakarsh Gupta
Mailing Address	Alta 101, 4735 21st Ave NE, Seattle, WA 98105
Phone	+1 (313) 663-0680
University Email	aakarshg@uw.edu
Personal Email	aakarshgupta919@gmail.com
GitHub	github.com/Aakarsh751
Project Repo	github.com/Aakarsh751/robstattm-pyport-AG-prop
LinkedIn	linkedin.com/in/aakarsh-gupta-128204218
Communication	Zoom, Google Meet
Time Zone	Pacific Time (PT, UTC−7)

2.2. Academic Affiliation

Institution	Degree	Period
University of Washington, Seattle	B.S. in Computational Finance and Risk Management (CFRM)	Aug 2025 – Mar 2027
University of Michigan	B.S. in Data Science *(Dean's List; International Undergraduate Scholar)*	Aug 2023 – Aug 2025

I am currently a **junior** with an expected graduation of **March 2027**. I began my undergraduate studies in Data Science at the University of Michigan, where I was placed on the Dean's List and was awarded the International Undergraduate Scholarship. In August 2025, I transferred to the University of Washington to pursue a B.S. in Computational Finance and Risk Management, a programme that sits at the intersection of mathematical statistics, numerical computation, and financial theory, which is exactly the disciplinary blend required to bridge rigorous robust-statistics research and the Python data-science ecosystem.

2.3. Relevant Coursework

The following UW courses form the academic backbone of this project:

Course	Title and Relevance
Completed	
CFRM 425	*R Programming for Quantitative Finance*: hands-on R package development and financial modelling in R
CFRM 410	*Probability & Statistics for Computational Finance*: estimation theory, asymptotic methods, sampling distributions
CFRM 405	*Mathematical Methods for Quantitative Finance*: optimisation, convexity, and numerical root-finding
AMATH 482	*Computational Methods for Data Analysis*: dimensionality reduction, PCA, and spectral methods in Python
AMATH 352	*Applied Linear Algebra & Numerical Analysis*: matrix decompositions and numerical stability
Spring 2026 (in progress)	
CFRM 421	*Machine Learning with Application to Finance*: Python-based ML methods applied to financial data, directly mirroring the use-case of `robstatpy`
CFRM 586	*Financial Time Series Forecasting Methods*: modelling heavy-tailed financial returns, a primary application domain for robust estimation

2.4. Technical Skills

- **Programming languages:** Python, R, SQL, C++
- **Python ecosystem:** NumPy, pandas, scikit-learn, matplotlib, PyTorch, TensorFlow, Selenium; REST API development with FastAPI and Streamlit
- **R ecosystem:** `ggplot2`, `tidyr`, `dplyr`, `RobStatTM`, `robustbase`; DataCamp-certified in R Programming, Data Manipulation with dplyr, and Data Visualisation with ggplot2
- **Development tools:** Git/GitHub, Jupyter, RStudio, Docker, Anaconda
- **Finance:** Bloomberg Finance Fundamentals; quantitative equity and crypto-market analysis; portfolio risk modelling

2.5. Relevant Experience and Projects

IT Risk Management and Analyst Intern, Virginia Department of Motor Vehicles
Spearheaded a proof-of-concept initiative to evaluate AI and NLP for analysing call-centre interactions, assessing customer sentiment, call-resolution efficiency, agent performance, and fraud-related risk indicators. Built FastAPI back-ends and Streamlit front-ends to deploy and interact with large language models, enabling real-time inference and operational decision support. Designed and automated intelligent workflows using n8n to streamline repetitive tasks, and processed unstructured speech data to produce cost-benefit analyses guiding in-house versus outsourced AI deployment decisions. This role directly exercised the API-design, packaging, and data-pipeline skills that are central to `robstatpy`.

Undergraduate Research Volunteer, University of Michigan – Imitation Learning
Conducted research on Imitation Learning under faculty guidance, training reinforcement-learning and human-guided agents with OpenAI Gym. Implemented the DAGGER and HG-DAGGER algorithms from the primary literature to improve agent performance, adaptability, and learning efficiency, and evaluated all models rigorously using PyTorch. This experience demonstrates the ability to read formal algorithm specifications, translate them into well-tested, reproducible code, and verify results against published benchmarks, a skill directly applicable to wrapping the MM- and S-estimation routines in `RobStatTM`.

Deep Learning Algorithm for Asthenopia Detection Independently conceived and developed a deep-learning image-classification pipeline for the early detection of asthenopia (visual fatigue). The project was recognised with a **Gold Award at the Indian Science and Engineering Fair** and achieved a top-100 national selection, providing evidence of the capacity to drive an independent research project from initial conception through to a peer-recognised outcome.

2.6. Why My Background Aligns with This Project

My training in Computational Finance and Risk Management (CFRM) is uniquely suited to this project. Robust statistics is not an abstract topic for me: it is central to the quantitative methods I am now beginning to study and apply. In CFRM 410 (Probability and Statistics), I learned estimation theory and asymptotic analysis that will allow me to understand the underpinnings of robust MM-estimators and S-Estimators that are covered in Maronna et al. (2019). In CFRM 425 (R Programming), I developed R packages and gained hands-on familiarity with the object system that `RobStatTM` uses internally. In CFRM 405 (Mathematical Methods), I studied the optimisation numerical algorithms that drive iterative robust estimators. AMATH 482 (Computational Methods for Data Analysis) gave me direct experience implementing PCA and dimensionality reduction in Python, the exact techniques that `pcaRobS` robustifies.

This combination means I understand *both* the R code base and its statistical foundations, and the Python ecosystem that `robstatpy` targets. I am not just a software engineer wrapping an unfamiliar library; I have the theoretical knowledge to verify correctness, interpret results, and extend the project into downstream applications such as robust portfolio construction and risk analysis (RPCRA).

2.7. Relationship with Mentors

I have been in active communication with **Prof. R. Douglas Martin** since **January 2026**, attending weekly meetings to discuss the project scope, the Python API design, and the technical challenges of bridging R's S3 object system with Python's language model. These conversations have directly shaped the implementation strategy presented in this proposal. I look forward to further engaging with the full mentorship team, Prof. Matias Salibian-Barrera and Brian G.

Peterson, as the project progresses.

2.8. Mentors

Name	Role	Contact
Prof. R. Douglas Martin	Primary mentor	martinrd3d@gmail.com
Prof. Matias Salibian-Barrera	Co-mentor	matias@stat.ubc.ca
Brian G. Peterson	Co-mentor	brian@braverock.com

2.9. Time Commitment and Availability

I am applying for the **large project size (350 hours)**. I am fully dedicated and available for the entire duration of the GSoC coding period (late May through mid-November 2026), with **30–35 hours per week** committed to `robstatpy` development. I will continue attending weekly meetings with Prof. Martin and will be responsive to all three mentors via email, Zoom, and GitHub.

3. Benefits to the Community

The robust-statistics methods described in `RobStatTM` including MM-regression, Rocke's S-estimator, robust PCA via M-scale minimization, among others, are the algorithmic companions to *Robust Statistics: Theory and Methods* (Maronna, Martin, Yohai & Salibian-Barrera, 2nd ed., 2019), a leading modern textbook in the field. Today, a Python practitioner who wants to reproduce a textbook example or apply these methods in their research or applications faces two unappealing choices: install and learn R, or re-implement algorithms from scratch in Python.

`robstatpy` removes this barrier by providing a pip[1]-installable Python package that:

1. **Exposes the full `RobStatTM` API** through `rpy2` wrappers, accepting NumPy arrays and pandas DataFrames and returning native Python objects.
2. **Guarantees numerical fidelity:** every output agrees with the R original to machine precision, because the R code itself executes underneath.
3. **Lowers the maintenance burden:** because the wrappers delegate to R, any upstream improvement to `RobStatTM` is automatically available in Python without duplicating development effort.
4. **Reproduces figures and numerical summaries** using `matplotlib` and `plotnine`[2] so that Python users can produce visualisations that closely mirror the R originals.
5. **Provides a comparative study** of wrapper-based versus direct-porting strategies, producing benchmarks and guidelines that will inform future R-to-Python interoperability projects.

Who benefits:

- *Data scientists, ML engineers, and Python-only practitioners* who need a rich toolkit of modern robust estimation methods without learning R. `robstatpy` provides a comprehensive,

[1] `pip` is the standard Python package installer. Running `pip install robstatpy` downloads and installs the package and all its dependencies automatically.

[2] `plotnine` is a Python implementation of R's `ggplot2` grammar-of-graphics, enabling near-identical plot syntax in Python. See https://plotnine.org/ and https://realpython.com/ggplot-python/.

self-contained Python library: users call familiar Python functions with NumPy arrays and pandas DataFrames and never need to write, read, or understand any R code.

- *Students and researchers* who study the Maronna et al. textbook and want to apply the methods in Jupyter rather than RStudio.

4. Current `RobStatTM` Functionality

The table below lists all `RobStatTM` R functions recommended in *Robust Statistics: Theory and Practice* (Maronna, Martin, Yohai, Salibián-Barrera, 2nd ed.), grouped by chapter. These form the complete target function set for `robstatpy`.

R Function	Description	Book §
Location & Scale (Chapter 2)		
`locScaleM`	Location and scale M-estimator (joint estimation)	2.3, 2.7
`mScale`	Scale M-estimator	2.5, 2.6
Linear Regression I (Chapter 4)		
`lmrobM`	M-estimator for regression with designed experiments	4.4
`rob.linear.test`	Robust likelihood-ratio test for linear hypotheses	4.7
Linear Regression II (Chapter 5)		
`lmrobdetMM`	MM-estimator with optimal ρ; primary regression recommendation	5.3, 5.9
`pyinit`	Highly robust Peña-Yohai initial estimator for `lmrobdetMM`	5.7
`step.lmrobdet`	Robust stepwise model selection via RFPE criterion	5.6
`lmrobdetDCML`	Distance-constrained MLE; boosts efficiency over MM	5.9
`pense`†	Robust MM elastic net (MM-ridge and MM-lasso as special cases)	5.1
Multivariate Analysis (Chapter 6)		
`covRobMM`	MM robust covariance estimator; recommended for $p < 10$	6.5
`covRobRocke`	Rocke S-type robust covariance estimator; recommended for $p \geq 10$	6.4.2, 6.4.4
`KurtSDNew`	Peña-Prieto highly robust initial covariance estimator	6.9.2
`pcaRobS`	Robust PCA based on a robust scale estimator	6.11.2
`GSE`†	Generalized S-estimator for covariance with missing data	6.12.2
`TSGS`†	Two-step GSE covariance estimator for cell-wise outliers	6.13
Generalized Linear Models (Chapter 7)		
`BYlogreg`††	M-estimator for logistic regression	7.2
`WBYlogreg`††	Redescending M-estimator for logistic regression (primary GLM recommendation)	7.2
`WMLlogreg`††	Weighted maximum-likelihood robust logistic regression	7.2

† Stretch deliverable (first priority): `pense`, `GSE`, and `TSGS` are not part of the `RobStatTM` package itself but are available in the `pense`, `GSE`, and `GSE` R packages respectively, and are recommended alongside `RobStatTM` functions in the book.

†† Stretch deliverable (second priority): robust GLM wrappers targeted for inclusion after core regression and multivariate coverage is complete.

5. GSoC Test Tasks and Validation

As part of the `robstatpy` GSoC 2026 application, two required test tasks were completed: implementing `rpy2` wrappers for `locScaleM` and `scaleM`, and validating their outputs against direct R calls. All 14 automated checks pass, confirming bit-identical numerical agreement between the Python wrappers and native R.

Test task notebook: robstatpy_comparison_rpy2.ipynb

Full wrapper code and result tables: robstatpy_tests.pdf

Selected numerical outputs — pure R vs. `robstatpy` (rpy2 wrapper):

Function	Output Field	Pure R	robstatpy
lmrobdetMM (mineral, mopt, eff = 0.95)			
	Intercept	15.2012	15.2012
	Slope (copper)	0.0126	0.0126
	Scale	9.9966	9.9966
	Robust R^2	0.0082	0.0082
covRobMM (wine, 13 variables, `set.seed(42)`)			
	Center[1..5]	13.75, 1.77, 2.46, 17.06, 106.19	13.75, 1.77, 2.46, 17.06, 106.19
	Max Mahalanobis dist	164.36	164.36
covRobRocke (wine, 13 variables, `set.seed(42)`)			
	Center[1..5]	13.73, 1.78, 2.47, 17.15, 106.77	13.73, 1.78, 2.47, 17.15, 106.77
	Max Mahalanobis dist	154.15	154.15
pcaRobS (bus, 3 components, `set.seed(42)`)			
	propSPC[1]	0.8611	0.8611
	propex	0.9731	0.9731

Differences between Pure R and `robstatpy` columns are zero or at machine epsilon ($\approx 4.4 \times 10^{-16}$) for all outputs; results are numerically identical because R executes underneath via `rpy2`.

6. Proposed Work

`robstatpy` exposes the complete `RobStatTM` function set to Python users via lightweight `rpy2` wrappers, providing numerically exact access to every estimator in the library. This section describes the wrapper approach and presents code examples for key functions already implemented.

6.1. Why Wrappers?

The `RobStatTM` package is **actively maintained** by its original authors, the same researchers who proved the theoretical properties of these estimators. A wrapper-based approach has three decisive advantages over full re-implementation:

1. **Exact numerical equivalence.** Because the R code executes underneath, Python users get bit-identical results. This is essential for textbook reproducibility and for cross-validating future pure-Python ports.
2. **Zero duplication of maintenance.** When the R authors fix a bug or add an estimator, the Python wrapper inherits the improvement immediately. A pure-Python re-implementation creates a second code base that must be independently maintained and validated.
3. **Speed of delivery.** Wrapping a function requires writing conversion glue, tests, and documentation, not re-deriving the algorithm. This makes it feasible to expose the *entire* API within a single GSoC period, rather than a subset.

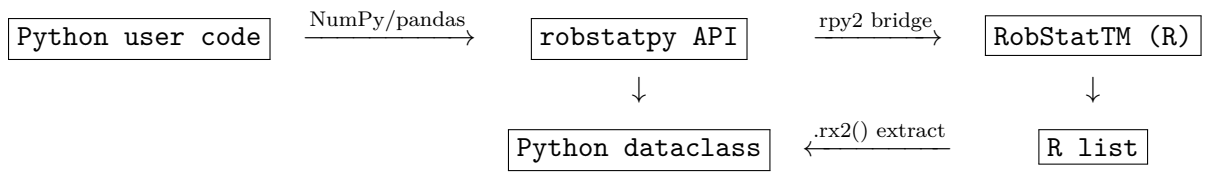
Together, these properties make the wrapper approach the most reliable path to bringing the full `RobStatTM` API to Python within a single GSoC period, without sacrificing any numerical accuracy.

6.2. `rpy2` Wrapper Layer

Every public `RobStatTM` function will be wrapped in a Python function that:

1. Accepts NumPy arrays and/or pandas DataFrames.
2. Converts inputs to R objects via `rpy2.robjects` and `numpy2ri/pandas2ri` converters.
3. Calls the R function inside `RobStatTM`.
4. Extracts named elements from the returned R list using `.rx2()`.
5. Returns a Python dataclass (or named dict) whose fields match the R return values.

Architecture diagram:



Key design decisions:

- **Global conversion context.** `rpy2` 3.6+ stores conversion rules in a `contextvars.ContextVar`, which is lost across Jupyter's async task boundaries. We patch this by calling `set_conversion(default_conv` at import time, ensuring conversions persist across cells.
- **R dot-to-Python underscore mapping.** `importr()` automatically maps `lmrobdet.control` to `lmrobdet_control`. We document both names in every docstring.
- **Lazy R package loading.** `RobStatTM` is loaded via `importr` only on first use, so importing `robstatpy` in a pure-Python context (e.g. for type hints or documentation builds) does not require R.

Code Example: `rpy2` Wrapper for `lmrobdetMM`

Listing 1: rpy2 wrapper for lmrobdetMM (regression; demonstrated with mopt family, efficiency=0.95)

```
def lmrobdet_mm(formula, data, family="mopt", efficiency=0.95):
    """Robust MM-regression via RobStatTM::lmrobdetMM().

    Parameters
    ----------
    formula : str
        R-style formula, e.g. 'zinc ~ copper'.
    data : pd.DataFrame
        Data frame (converted to R via pandas2ri).
    family : str
        Psi function family for IRLS refinement.
    efficiency : float
        Target asymptotic efficiency.

    Returns
    -------
    dict with 'coefficients', 'scale', 'residuals',
         'fitted_values', 'rweights', 'r_squared'.
    """
    from rpy2.robjects import pandas2ri
    pandas2ri.activate()
    r_df = ro.conversion.py2rpy(data)
    ctrl = robstattm.lmrobdet_control(
        family=family, efficiency=efficiency)
    fit = robstattm.lmrobdetMM(
        ro.Formula(formula), data=r_df, control=ctrl)
    return {
        "coefficients":  np.array(fit.rx2("coefficients")),
        "scale":         fit.rx2("scale")[0],
        "residuals":     np.array(fit.rx2("residuals")),
        "fitted_values": np.array(fit.rx2("fitted.values")),
        "rweights":      np.array(fit.rx2("rweights")),
        "r_squared":     fit.rx2("r.squared")[0],
    }
```

7. Alternative Implementation Strategies

Two additional approaches were implemented as cross-checks and to demonstrate different trade-offs in R–Python interoperability. Both are described briefly here; full code is available in the project notebooks.

7.1. Strategy B: Subprocess + JSON Wrappers

As a comparison baseline, the project also implements wrappers using `subprocess` calls to `Rscript` with `jsonlite`-serialised results. This approach:

- Requires no compiled `rpy2` dependency (useful on systems where building `rpy2` is difficult).
- Incurs higher per-call overhead due to R process startup.
- Serves as a cross-check: if both wrappers produce identical outputs, we have high confidence in the conversion layer.

7.2. Strategy C: Direct Python Re-implementation

For `locScaleM` and `scaleM`, the project provides pure-Python re-implementations to serve as the GSoC test tasks and to establish a methodology for future direct porting.

8. Evaluation Plan: Comparative Benchmarks

A central deliverable is a rigorous comparison of the three implementation strategies across three dimensions:

8.1. Metrics

1. **Accuracy.** For each function, compute $\max_i |y_i^{\mathrm{Py}} - y_i^{\mathrm{R}}|$ across all return fields, for both the rpy2 wrapper (expected: 0) and the pure-Python port (expected: $< 10^{-6}$).
2. **Speed.** Wall-clock time per call, measured with `timeit` over 100 repetitions on synthetic data of sizes $n \in \{100, 1000, 10000\}$ and $p \in \{5, 20, 50\}$. Breakdown: R computation time vs. conversion overhead.
3. **Coverage.** Number of target functions wrapped and passing all validation checks, tracked against the 18-function target set in Section 4 (e.g., 6/18 at proposal time; 18/18 as the final deliverable).

8.2. Benchmark Protocol

- **Datasets:** Both synthetic and real datasets are used. Synthetic samples are drawn from a Gaussian base distribution with $\varepsilon = 5\%$ contamination — 5% of observations are replaced by outliers placed 10σ from the mean — to stress-test breakdown behaviour across sample sizes $n \in \{100, 1000, 10000\}$ and dimensions $p \in \{5, 20, 50\}$. Real datasets from the book (mineral, wine, bus) are used to reproduce published results.
- **Functions benchmarked:** `locScaleM`, `scaleM`, `lmrobdetMM`, `covRobMM`, `covRobRocke`, `pcaRobS`.
- **Accuracy comparison:** Each `robstatpy` wrapper output is compared field-by-field against a direct R session call (same data, same seed) to confirm zero numerical difference. Where a `scikit-learn` or `statsmodels` equivalent exists, results are also compared to quantify the practical difference between robust and classical estimators.
- **Speed comparison:** Wall-clock time per call measured with `timeit` over 100 repetitions. Comparison grid: native R session vs. `rpy2` wrapper (showing bridge overhead) vs. subprocess wrapper vs. nearest Python equivalent.
- **Reporting:** Results presented in a Jupyter notebook with timing tables, accuracy comparison tables, and commentary.

9. Implementation Plan and Timeline

The project follows a **22-week large-project timeline** (350 hours), structured around the GSoC 2026 calendar. Testing and documentation are treated as first-class deliverables and are built incrementally within each phase rather than deferred.

9.1. Community Bonding Period (May 1 – May 24)

- Finalise package scaffolding: `pyproject.toml`, `pytest` configuration, CI/CD with GitHub Actions (R + Python matrix).
- Study `RobStatTM` vignettes and the 26 example R scripts in the project repository to understand expected inputs, outputs, and edge cases for each function.

- Draft the `robstatpy` API surface: naming conventions, type stubs, and docstring templates modelled on R man pages.
- Set up Sphinx + ReadTheDocs skeleton; agree on code review cadence and milestone criteria with mentors.

9.2. Phase 1: Univariate Wrappers (Weeks 1–3, May 25 – Jun 14)

Week	Deliverables
1	`locScaleM` rpy2 wrapper (bisquare, huber, mopt families; eff $\in \{0.85, 0.90, 0.95\}$). Full `pytest` suite verifying field-by-field agreement with R. Docstring and API docs entry.
2	`mScale` rpy2 wrapper. Document relationship between `mScale` and `locScaleM`. Subprocess wrapper as numerical cross-check. Pytest suite.
3	`lmrobdet.control` wrapper exposing all tuning parameters. Reproduce Chapter 2 textbook examples in a Jupyter notebook. Integration tests for the complete univariate module.

9.3. Phase 2: Robust Regression (Weeks 4–7, Jun 15 – Jul 5)

Week	Deliverables
4	`lmrobdetMM` wrapper: formula interface, coefficient extraction, residuals, robustness weights, scale. Pytest suite; docstring.
5	`lmrobdetMM` diagnostics: robust R^2, fitted values, summary table mirroring `summary()` output. `lmrobdetDCML` wrapper + pytest.
6	`step.lmrobdet` wrapper for stepwise model selection. `pyinit` and `rob.linear.test` wrappers. Reproduce mineral dataset examples (Figs 5.1–5.7).
7	**Midterm milestone:** regression module complete with $\geq 90\%$ pytest coverage. Docstrings for all regression wrappers. Midterm evaluation report.

Midterm Evaluation (Jul 6–10): Deliverables: univariate and regression wrappers (7 functions), pytest suite, Chapter 2 and 5 reproduction notebooks, $\geq 90\%$ test coverage on delivered modules.

9.4. Phase 3: Robust Covariance and PCA (Weeks 8–11, Jul 13 – Aug 9)

Week	Deliverables
8	`covRobMM` wrapper: center, covariance, correlation, Mahalanobis distances, weights. Pytest suite; docstring.
9	`covRobRocke` and `KurtSDNew` wrappers. Reproduce wine dataset examples (Fig 6.3). Pytest suite.
10	`pcaRobS` wrapper: eigenvectors, proportion of variance, scores. Reproduce bus dataset examples (Fig 6.10). Pytest suite.
11	Distance–distance plot and scree plot utility functions. Documentation pass for all multivariate wrappers. End-to-end integration tests for covariance and PCA module.

9.5. Phase 4: Benchmarks and Comparative Study (Weeks 12–13, Aug 10 – Aug 23)

Week	Deliverables
12	Speed benchmarks: native R session vs. rpy2 wrapper vs. subprocess wrapper vs. `scikit-learn`/`statsmodels` baselines. Synthetic contaminated Gaussian data generator.
13	Accuracy benchmarks: max absolute error across all functions and parameter combinations. Tabulate rpy2 bridge overhead. Comparative Jupyter notebook with timing tables, accuracy results, and commentary.

9.6. Phase 5: Documentation, Packaging, and Tutorials (Weeks 14–17, Aug 24 – Sep 20)

Week	Deliverables
14	Complete Sphinx API documentation for all delivered wrappers. Every function has a docstring modelled on the R man page, with usage examples. Publish on ReadTheDocs.
15	Publish `robstatpy` on PyPI (sdist + wheel). Installation guide covering R and `rpy2` setup on Linux/macOS/Windows with troubleshooting notes.
16	Tutorial Jupyter notebooks reproducing key textbook examples from Chapters 2, 5, and 6 of Maronna et al. (2019).
17	Final integration tests across all modules end-to-end. Contributor guide and README polish.

9.7. Phase 6: Stretch Wrappers and Final Submission (Weeks 18–22, Sep 21 – Nov 2)

Week	Deliverables
18–19	**GLM wrappers (stretch):** `BYlogreg`, `WBYlogreg`, `WMLlogreg` wrappers for robust logistic regression. Pytest suite and docstrings. Reproduce Chapter 7 examples.
20	**Additional stretch wrappers:** `lmrobM` (regression for designed experiments). `pense`, `GSE`, `TSGS` wrappers where time permits.
21	Final documentation review: ensure all wrappers are covered, all examples run, all tests pass. Address outstanding mentor feedback.
22	**Final submission:** Complete code review, final evaluation report, archive of all notebooks and benchmarks.

10. Deliverables

10.1. Required Deliverables

1. **`robstatpy` Python package** (pip-installable) exposing the following `RobStatTM` functions via lightweight `rpy2` wrappers:
   - `locScaleM`, `mScale` (univariate location and scale)
   - `lmrobdetMM`, `lmrobdetDCML`, `lmrobdet.control`, `step.lmrobdet`, `pyinit`, `rob.linear.test` (regression)
   - `covRobMM`, `covRobRocke`, `KurtSDNew` (robust covariance)
   - `pcaRobS` (robust PCA)

2. **Pytest test suite** achieving $\geq 90\%$ coverage across all wrapper functions, with field-by-field comparison against direct R calls confirming zero numerical difference for every output.

3. **Comparative benchmark notebook** evaluating speed (native R vs. rpy2 vs. subprocess) and accuracy across all wrapped functions, on both synthetic contaminated data and real book datasets (mineral, wine, bus).

4. **Sphinx API documentation** published on ReadTheDocs, with docstrings modelled on the `RobStatTM` R man pages and runnable usage examples for each function.

5. **Tutorial Jupyter notebooks** reproducing key textbook examples from Chapters 2, 5, and 6 of Maronna et al. (2019), demonstrating end-to-end workflows in Python.

10.2. Stretch Deliverables

1. **GLM wrappers** (first priority): `BYlogreg`, `WBYlogreg`, `WMLlogreg` for robust logistic regression (Chapter 7); `lmrobM` for regression with designed experiments (Chapter 4). Pytest suite and docstrings for each.

2. **External package wrappers** (first priority): `pense` (robust MM elastic net), `GSE` (generalised S-estimator for missing data), and `TSGS` (two-step cell-wise outlier estimator) — recommended alongside `RobStatTM` functions in the book but residing in separate R packages.

3. **Complete native Python port** of the remaining `RobStatTM` functions (`lmrobdetMM`, `covRobMM`, `covRobRocke`, `pcaRobS`), validated against the rpy2 wrappers, with optional `numba` JIT acceleration for performance-critical inner loops.

11. Distribution and Packaging

`robstatpy` will be distributed through two complementary channels to maximise accessibility:

11.1. GitHub Installation (Development and Bleeding-Edge)

The package will be hosted on GitHub under an open-source license (MIT or Apache 2.0, to be confirmed with mentors). Users can install directly from the repository:

Listing 2: GitHub installation

```
pip install git+https://github.com/Aakarsh751/robstatpy.git
```

This channel provides access to the latest development version and makes it straightforward for contributors to fork, modify, and submit pull requests. The repository will include:

- A `pyproject.toml` with full metadata and dependency specifications (`rpy2 >= 3.5`, `numpy`, `pandas`, `scipy`, `matplotlib`).
- GitHub Actions CI running the full `pytest` suite on Linux, macOS, and Windows against multiple R versions (4.3, 4.4, 4.5).
- A `CONTRIBUTING.md` with guidelines for adding new wrappers or submitting pure-Python ports.
- Tagged releases corresponding to each PyPI publication.

11.2. PyPI Publication (Stable Releases)

Stable, versioned releases will be published on the Python Package Index (PyPI), enabling standard installation:

Listing 3: PyPI installation

```
pip install robstatpy
```

The packaging workflow uses:

- `build` to produce source distributions (`sdist`) and pure-Python wheels.
- `twine` for secure upload to PyPI.
- Semantic versioning (e.g., `0.1.0` for the initial GSoC release, `0.2.0` after stretch wrappers are added).
- A GitHub Actions release workflow that automatically publishes to PyPI when a new git tag is pushed.

R dependency handling: Since `robstatpy` requires a working R installation with `RobStatTM`, the package will: (1) detect `R_HOME` automatically at import time, (2) provide a clear `ImportError` message with installation instructions if R is not found, and (3) include a `robstatpy.check_setup()` utility that validates the R environment and prints version information. The installation guide will cover R setup on all three major platforms (Linux via `apt/conda`, macOS via Homebrew, Windows via the CRAN installer).

12. Risks and Mitigation Strategies

Risk	Impact	Mitigation
`rpy2` build failures on Windows/macOS	Delays CI and user installation	Provide pre-built wheels where possible; subprocess wrapper as independent fallback; Docker image for reproducible builds
R or `RobStatTM` version incompatibility	Wrapper outputs may differ across R versions; breaking API changes could invalidate tests	Pin minimum R version (4.3+) and `RobStatTM` version; test CI matrix against R 4.3, 4.4, 4.5; add version check at import time
Complex rpy2 type conversion edge cases	Some `RobStatTM` functions return nested R lists or named vectors that do not map cleanly to Python objects	Each wrapper undergoes field-by-field validation against direct R output; conversion helpers (`rx`, `r2py`) are tested independently before use
Scope of 11 core functions proving too ambitious	Later phases (multivariate, PCA) could be under-delivered	Phases are ordered by dependency; univariate and regression wrappers (Phases 1–2) are self-contained and sufficient for a strong midterm; stretch functions absorb any schedule risk
Mentor availability during summer	Reduced feedback cadence	Three mentors provide redundancy; asynchronous communication via GitHub issues; weekly written status updates regardless

13. References

[1] R. A. Maronna, R. D. Martin, V. J. Yohai, and M. Salibian-Barrera. *Robust Statistics: Theory and Methods (with R)*, 2nd ed. Wiley, 2019.

[2] M. Salibian-Barrera, G. Willems, and R. Zamar. "The fast-τ estimator for regression." *Journal of Computational and Graphical Statistics*, 2008.

[3] V. J. Yohai. "High breakdown-point and high efficiency robust estimates for regression." *The Annals of Statistics*, 15(2):642–656, 1987.

[4] D. L. Rocke. "Robustness properties of S-estimators of multivariate location and shape in high dimension." *The Annals of Statistics*, 24(3):1327–1345, 1996.

[5] L. Gautier et al. `rpy2`: R in Python. https://rpy2.github.io/, 2024.

[6] R. D. Martin. `PCRA`: Portfolio Construction and Risk Analysis. CRAN, 2023.